Logistic Regression Weight Optimization

Yunui Kang
Department of Energy Engineering
Korea Institute of Energy Technology, KENTECH
Naju, South Korea
yunuikang@kentech.ac.kr

***Abstract*—Standard optimization techniques in machine learning, such as Gradient Descent and Newton's Method, typically operate in unconstrained environments, often neglecting the structural or logical bounds required in high-stakes domains like medical diagnosis. This paper presents the design, implementation, and validation of a custom Nonlinear Constrained Optimization Solver based on the Interior Point Method (IPM). By integrating a logarithmic barrier function with a second-order Newton engine, the proposed solver transforms constrained optimization problems into a sequence of unconstrained sub-problems, allowing for the rigorous enforcement of inequality constraints without compromising convergence speed. The methodology was validated through a two-phase experimentation process: a synthetic 2D toy project for architectural verification and a real-world application using the 31-dimensional UCI Breast Cancer Wisconsin dataset. In the latter, medical domain knowledge regarding the correlation between tumor characteristics and malignancy was encoded as inequality constraints on feature weights. The results demonstrate that the solver achieves quadratic convergence comparable to unconstrained Newton methods and strictly satisfies the Karush-Kuhn-Tucker (KKT) optimality conditions. Notably, the constrained model achieved a test set accuracy of 98.25%, surpassing its training accuracy of 97.80%. These findings suggest that integrating domain-driven constraints acts as a powerful form of regularization, significantly enhancing model generalization and interpretability on unseen data.**

***Index Terms*—Constrained Optimization, Interior Point Method, Logistic Regression, Medical Diagnosis**

I. Introduction

Optimization lies at the heart of machine learning, serving as the mathematical engine that trains models to recognize patterns and make predictions [1]. While standard unconstrained optimization techniques such as Gradient Descent and Newton's Method are ubiquitous in minimizing loss functions, real-world applications often demand adherence to specific structural or logical constraints [2]. In domains such as medical diagnosis, physics-informed machine learning, or finance, model parameters must often satisfy bounds derived from domain knowledge (e.g., a tumor's size must positively correlate with malignancy probability) to ensure interpretability and reliability [3].

This project presents the design, implementation, and validation of a custom nonlinear constrained optimization solver based on the Interior Point Method (IPM). By integrating a logarithmic barrier function with a second-order Newton engine, we transform the constrained optimization problem into a sequence of unconstrained sub-problems. This architecture allows rigorous enforcement of constraints while leveraging the quadratic convergence speed of Newton's method.

The study is structured into two phases. First, a *toy project* utilizes a synthetic 2D dataset to establish the mathematical foundation, validate the solver architecture, and visualize the optimization landscape. Second, the methodology is scaled to a *real-world application* using the high-dimensional UCI Breast Cancer Wisconsin dataset [4]. Here, medical domain knowledge is encoded into the model as inequality constraints on 30 feature weights.

Through analysis and assessment, this report demonstrates that integrating domain constraints not only ensures mathematical validity but also acts as a powerful regularizer, enhancing the model's generalization capabilities on unseen data.

II. Theoretical Background

A. Logistic Regression and the Sigmoid Activation

The core objective of this project is binary classification, where the goal is to predict the probability $p(y = 1 \mid \mathbf{x})$ that a given input vector $\mathbf{x}$ belongs to the positive class [5]. A standard linear model,

$$z = \mathbf{w}^\top \mathbf{x} + b, \tag{1}$$

is unsuitable for this task because its output range $(-\infty, \infty)$ violates the axioms of probability, which require values to lie strictly within $[0, 1]$.

To resolve this, logistic regression employs the sigmoid function (also known as the logistic function) as a non-linear activation function. The sigmoid function $\sigma(z)$ is defined as [5]:

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \tag{2}$$

The selection of the sigmoid function is critical for three mathematical reasons:

- *Probabilistic mapping:* It maps any real-valued number into the range $(0, 1)$, allowing the output to be interpreted directly as a probability [5].
- *Differentiability:* The function is smooth and differentiable everywhere. Crucially, its derivative can be expressed in terms of the function itself,

  $$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big), \tag{3}$$

  which simplifies the computation of gradients and Hessians required for Newton's Method.

- *Decision boundary:* It establishes a clear linear decision boundary at $z = 0$ (where $\sigma(0) = 0.5$), making the model interpretable.

B. Maximum Likelihood Estimation and the Objective Function

To estimate the optimal parameters $\mathbf{w}$, we adopt the principle of Maximum Likelihood Estimation (MLE). For a binary outcome following a Bernoulli distribution, the likelihood of observing the training data $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^{N}$ given weights $\mathbf{w}$ is [1]:

$$L(\mathbf{w}) = \prod_{i=1}^{N} p_i^{y_i} \left(1 - p_i\right)^{1-y_i}, \tag{4}$$

where $p_i = \sigma(\mathbf{w}^\top \mathbf{x}_i)$. Maximizing this product directly is computationally unstable due to the risk of arithmetic underflow. Therefore, we take the natural logarithm to convert the product into a sum, resulting in the log-likelihood. Since optimization algorithms typically seek to minimize a cost function rather than maximize a utility, we negate the expression to define the Negative Log-Likelihood (NLL) loss function:

$$f(\mathbf{w}) = -\sum_{i=1}^{N} \left[y_i \log(p_i) + (1 - y_i) \log\left(1 - p_i\right)\right]. \tag{5}$$

The NLL was chosen over other loss functions (such as Mean Squared Error) for two reasons pertinent to this study:

- *Convexity:* When combined with the sigmoid activation, the NLL function is convex with respect to $\mathbf{w}$. This guarantees that any local minimum found by the solver is the global minimum, ensuring the reliability of the optimization process [6].
- *Severe penalization:* The logarithmic form imposes an asymptotic penalty on severe errors. If the model confidently predicts the wrong class (e.g., $p_i \to 0$ when $y_i = 1$), the loss approaches infinity ($-\log(0) \to \infty$). This forces the solver to prioritize correcting severe misclassifications, leading to more robust decision boundaries.

III. Toy Project: Concept Verification and Solver Architecture Design

This section details the design and verification of a custom nonlinear constrained optimization solver using a synthetic dataset. The primary objective is to establish a mathematical foundation for the solver and validate the implementation of the Interior Point Method (IPM) before applying it to high-dimensional real-world data.

A. Problem Formulation and Mathematical Modeling

1) Data Generation: To facilitate direct visualization and debugging of the optimization landscape, a two-dimensional synthetic binary classification problem was formulated. Two distinct classes were generated using multivariate Gaussian distributions to simulate a binary classification scenario:

- *Class 0:* 15 samples generated from $\mathcal{N}(\boldsymbol{\mu} = [1, 1]^\top, \sigma^2\mathbf{I})$ with $\sigma = 0.5$.

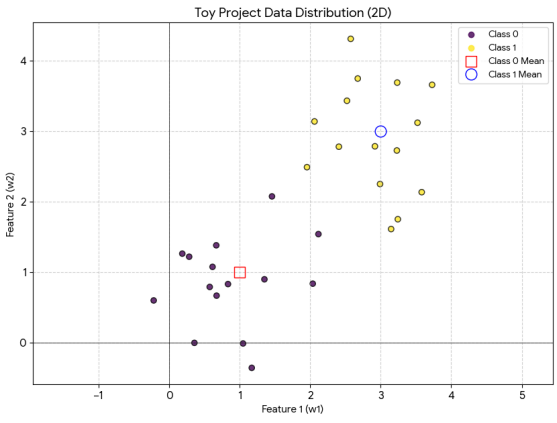


Fig. 1. Feature distribution of the generated 2D toy dataset.

- *Class 1:* 15 samples generated from $\mathcal{N}(\boldsymbol{\mu} = [3, 3]^\top, \sigma^2\mathbf{I})$ with $\sigma = 0.5$.
- *Bias augmentation:* To incorporate the bias term w_b into the weight vector, a constant value of 1 was appended to all feature vectors, yielding

$$\tilde{\mathbf{x}}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \\ 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ w_b \end{bmatrix}. \tag{6}$$

2) Objective Function: Negative Log-Likelihood: The optimization objective was defined as the Negative Log-Likelihood (NLL) of a logistic regression model. The objective function $f(\mathbf{w})$ is expressed as:

$$f(\mathbf{w}) = -\sum_{i=1}^{N} \Big[y_i \log\big(\sigma(\tilde{\mathbf{x}}_i^\top \mathbf{w})\big) + (1 - y_i) \log\big(1 - \sigma(\tilde{\mathbf{x}}_i^\top \mathbf{w})\big)\Big], \tag{7}$$

where $\sigma(\cdot)$ denotes the sigmoid activation function.

The motivation for minimizing (7) can be understood by considering two conditional cases:

a) Case 1 ($y_i = 1$): The objective reduces to minimizing $-\log(p_i)$. If the model correctly predicts a high probability ($p_i \to 1$), the loss approaches $-\log(1) = 0$. Conversely, if the model incorrectly predicts a low probability ($p_i \to 0$), the loss approaches $-\log(0) \to \infty$, imposing an infinite penalty on the error.

b) Case 2 ($y_i = 0$): The objective reduces to minimizing $-\log(1 - p_i)$. If the model correctly predicts a low probability ($p_i \to 0$), the loss approaches $-\log(1) = 0$. If the model incorrectly predicts a high probability ($p_i \to 1$), the loss approaches infinity.

This formulation ensures that the optimization process is driven by severe penalties for confident but incorrect predictions.

3) Constraints: Semantic Reasoning: To test the solver's capability to handle inequality constraints, we imposed a non-

negativity constraint on the feature weights while leaving the bias term unconstrained. This is formulated as

$$w_1 \geq 0, \quad w_2 \geq 0, \tag{8}$$

or equivalently in standard inequality form:

$$g_1(\mathbf{w}) = -w_1 \leq 0, \quad g_2(\mathbf{w}) = -w_2 \leq 0, \tag{9}$$

where w_b is unconstrained [7].

B. Development of the Unconstrained Optimization Engine

Before addressing the inequality constraints, a robust computational engine was developed to solve unconstrained subproblems:

$$\min_{\mathbf{w}} \; f(\mathbf{w}). \tag{10}$$

Three distinct optimization algorithms—Gradient Descent (GD), Newton's Method, and BFGS—were implemented and rigorously compared. The goal was to identify the most efficient solver to serve as the *inner solver* for the subsequent constrained optimization stage.

1) Backtracking Line Search: To ensure global convergence and numerical stability, a backtracking line search procedure was integrated into all three solvers, replacing fixed learning rates. Instead of taking a step of size $\alpha = 1.0$, the algorithm iteratively reduces the step size

$$\alpha \leftarrow \tau\alpha, \quad \tau = 0.5, \tag{11}$$

until the Armijo (sufficient decrease) condition is satisfied [1], [7]:

$$f(\mathbf{w} + \alpha\mathbf{d}) \leq f(\mathbf{w}) + \beta\alpha\big(\nabla f(\mathbf{w})^\top \mathbf{d}\big). \tag{12}$$

The function initializes $\alpha = 1.0$. If the new loss does not decrease sufficiently relative to the gradient slope, α is reduced. This prevents the solver from overshooting the minimum or oscillating unstably [7].

2) Gradient Descent (GD): Gradient Descent is a first-order optimization algorithm that iteratively updates weights in the opposite direction of the gradient [7].

- *Gradient calculation:* At the current position $\mathbf{w}_k$, compute $\mathbf{g}_k = \nabla f(\mathbf{w}_k)$.
- *Convergence check:* Terminate if the L_2-norm $\|\mathbf{g}_k\|_2$ falls below a tolerance (e.g., 10^{-6}), indicating a (near-)stationary point.
- *Direction:* Set the descent direction $\mathbf{d}_k = -\mathbf{g}_k$ (steepest descent).
- *Update:* Update weights using the step size α from line search:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha\mathbf{d}_k. \tag{13}$$

However, GD relies solely on first-order information. In ill-conditioned problems ("narrow valleys"), the gradient direction may be nearly orthogonal to the direction of the minimum, causing inefficient zigzagging and slow, linear convergence [7].

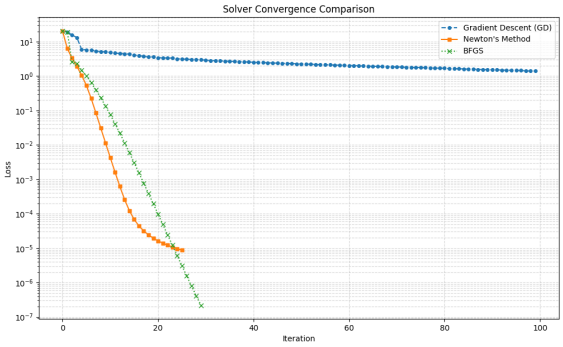


Fig. 2. Convergence comparison of GD, Newton's Method, and BFGS on the toy problem.

3) Newton's Method: Newton's Method is a second-order optimization technique that leverages curvature information to achieve rapid convergence.

- *Hessian calculation:* Compute the Hessian $\mathbf{H}_k = \nabla^2 f(\mathbf{w}_k)$.
- *Newton step:* Solve the linear system

$$\mathbf{H}_k\mathbf{d}_k = -\mathbf{g}_k, \tag{14}$$

  which is commonly written as $\mathbf{d}_k = -\mathbf{H}_k^{-1}\mathbf{g}_k$.
- *Update:* Update $\mathbf{w}_{k+1} = \mathbf{w}_k + \alpha\mathbf{d}_k$ with α determined by line search.

By accounting for curvature, Newton's method mitigates the zigzagging behavior of GD. Near the optimum, it exhibits quadratic convergence, reaching high precision with very few iterations [7].

4) BFGS (Broyden–Fletcher–Goldfarb–Shanno): BFGS is a quasi-Newton method that approximates the inverse Hessian, avoiding explicit Hessian inversion.

- *Initialization:* Start with an initial inverse-Hessian approximation, typically $\mathbf{Q}_0 = \mathbf{I}$.
- *Direction calculation:* Compute $\mathbf{d}_k = -\mathbf{Q}_k\mathbf{g}_k$.
- *Displacement vectors:* After updating $\mathbf{w}_{k+1}$, compute

$$\boldsymbol{\delta}_k = \mathbf{w}_{k+1} - \mathbf{w}_k, \quad \boldsymbol{\gamma}_k = \mathbf{g}_{k+1} - \mathbf{g}_k. \tag{15}$$

- *Matrix update:* Update $\mathbf{Q}$ using the rank-2 update:

$$\mathbf{Q}_{k+1} = \left(\mathbf{I} - \frac{\boldsymbol{\delta}_k\boldsymbol{\gamma}_k^\top}{\boldsymbol{\delta}_k^\top\boldsymbol{\gamma}_k}\right)\mathbf{Q}_k\left(\mathbf{I} - \frac{\boldsymbol{\gamma}_k\boldsymbol{\delta}_k^\top}{\boldsymbol{\delta}_k^\top\boldsymbol{\gamma}_k}\right) + \frac{\boldsymbol{\delta}_k\boldsymbol{\delta}_k^\top}{\boldsymbol{\delta}_k^\top\boldsymbol{\gamma}_k}. \tag{16}$$

A key feature of BFGS is that if $\mathbf{Q}_k$ is positive definite, then $\mathbf{Q}_{k+1}$ remains positive definite provided $\boldsymbol{\delta}_k^\top\boldsymbol{\gamma}_k > 0$, ensuring $\mathbf{d}_k$ is a valid descent direction. In the implementation, if the denominator $\rho = \boldsymbol{\delta}_k^\top\boldsymbol{\gamma}_k$ is close to zero (indicating minimal curvature change), the update is skipped to prevent numerical instability [7].

5) Comparative Analysis and Selection: As expected, GD exhibited slow convergence, requiring over 100 iterations to significantly reduce the loss, confirming its inefficiency

in navigating the curvature of the NLL objective. In contrast, Newton and BFGS demonstrated superior performance. Newton's method converged almost vertically, showcasing its quadratic convergence rate. BFGS also reached the optimal solution (loss ≈ 0) rapidly, validating its effectiveness as a computationally efficient alternative to Newton's method.

Conclusion: Based on these results, Newton's Method (and alternatively BFGS) was selected as the inner engine for the constrained solver, as its ability to leverage curvature is essential for navigating the landscape introduced by logarithmic barrier functions.

C. Constrained Solver Architecture: Interior Point Method

While the Newton engine developed in the previous phase efficiently minimizes the objective function $f(\mathbf{w})$, it operates as an unconstrained solver and therefore disregards feasibility boundaries defined by $g(\mathbf{w}) \leq 0$. If the global minimum of the unconstrained negative log-likelihood lies in a region where some $w_i < 0$, the solver will violate the domain-knowledge constraints.

To enforce the non-negativity constraints ($w_i \geq 0$ for feature weights) while retaining the convergence properties of Newton's method, we implemented an Interior Point Method (IPM) architecture. Conceptually, this approach erects an "infinite potential barrier" along the boundary of the feasible region ($w_i = 0$), mathematically preventing the solver from crossing into infeasible space [7].

1) The Logarithmic Barrier Function: The constrained optimization problem was transformed into a sequence of unconstrained problems by augmenting the original objective $f(\mathbf{w})$ with a logarithmic barrier term. Using a barrier parameter $\rho > 0$, the surrogate objective $L(\mathbf{w}, \rho)$ is defined as

$$L(\mathbf{w}, \rho) = f(\mathbf{w}) - \frac{1}{\rho} \left(\log w_1 + \log w_2 \right), \tag{17}$$

where the barrier term $-(\log w_1 + \log w_2)$ penalizes proximity to the boundary. As any weight w_i approaches zero (e.g., $0.1, 0.01, \ldots$), $\log(w_i) \to -\infty$, and therefore $-\frac{1}{\rho}\log(w_i) \to +\infty$. Since the solver minimizes $L(\mathbf{w}, \rho)$, this infinite penalty forces the optimization trajectory to remain strictly within the interior of the feasible region ($w_i > 0$).

2) Modification of Derivatives: To utilize the second-order Newton engine, the gradient and Hessian of the augmented function $L(\mathbf{w}, \rho)$ were derived analytically. Let $\nabla f(\mathbf{w})$ and $\nabla^2 f(\mathbf{w})$ denote the gradient and Hessian of the original objective. Then,

$$\nabla L(\mathbf{w}, \rho) = \nabla f(\mathbf{w}) - \frac{1}{\rho} \begin{bmatrix} \frac{1}{w_1} \\ \frac{1}{w_2} \\ 0 \end{bmatrix}, \tag{18}$$

which introduces a repulsive force that grows as w_i approaches 0. Similarly, the Hessian becomes

$$\nabla^2 L(\mathbf{w}, \rho) = \nabla^2 f(\mathbf{w}) + \frac{1}{\rho} \operatorname{diag}\left(\frac{1}{w_1^2}, \frac{1}{w_2^2}, 0 \right), \tag{19}$$

TABLE I
CONVERGENCE OF THE CONSTRAINED SOLVER (TOY PROBLEM).

Outer Iter	ρ	Barrier/ρ	w_1	w_2	w_b
0	10^0	-1.28×10^1	568.7882	657.1933	-2224.8318
1	10^1	-1.29×10^0	582.0316	686.9637	-2299.7019
2	10^2	-1.29×10^{-1}	582.0316	686.9637	-2299.7019
3	10^3	-1.29×10^{-2}	582.0316	686.9637	-2299.7019
4	10^4	-1.29×10^{-3}	582.0316	686.9637	-2299.7019
5	10^5	-1.29×10^{-4}	582.0316	686.9637	-2299.7019
6	10^6	-1.29×10^{-5}	582.0316	686.9637	-2299.7019
7	10^7	-1.29×10^{-6}	582.0316	686.9637	-2299.7019

creating extreme curvature near the boundary, which naturally encourages smaller Newton steps as the iterate approaches the constraints.

3) Feasibility-Aware Line Search: A standard backtracking line search is insufficient because the logarithmic barrier is undefined for $w_i \leq 0$. Therefore, a feasibility check was integrated into the line search.

Given a candidate update $\mathbf{w}_{\text{new}} = \mathbf{w} + \alpha \mathbf{d}$, if any constrained component violates feasibility,

$$(\mathbf{w}_{\text{new}})_i \leq 0 \quad \text{for any constrained } i, \tag{20}$$

the step size α is immediately rejected and reduced (without evaluating the objective). This mechanism guarantees the "interior point" property throughout optimization.

4) Sequential Unconstrained Minimization (Outer Loop): The solver is orchestrated by an *outer loop* that solves a sequence of unconstrained barrier sub-problems while updating the barrier parameter ρ:

- *Initialization:* Start with a small parameter (e.g., $\rho = 1$). The barrier is strong, and the solver finds a safe optimum $\mathbf{w}^*(\rho)$ well inside the feasible region.
- *Iterative weakening:* Increase ρ geometrically (e.g., $\rho_{k+1} = 10\rho_k$). This reduces the barrier weight $1/\rho$, allowing the solution to approach the boundary if the unconstrained optimum lies near it.
- *Convergence:* As $\rho \to \infty$, $L(\mathbf{w}, \rho)$ approaches $f(\mathbf{w})$ within the feasible region. The outer loop terminates when the barrier influence $1/\rho$ falls below a tolerance ϵ, yielding the final constrained optimum $\mathbf{w}^*$.

5) Optimization Results: Table I summarizes the outer-loop convergence behavior. The barrier influence decreased from -1.28×10^1 to -1.29×10^{-6}, becoming effectively negligible, while the weights stabilized.

As shown in Table I, the solver converged successfully. The weights stabilized at $\mathbf{w}^* = [582.03,\ 686.96,\ -2299.70]^\top$ with a final loss of 0.000342.

D. Validation and Analysis

To ensure the reliability of the proposed solver, the obtained solution $\mathbf{w}^*$ was subjected to a two-step verification process: (i) mathematical correctness via KKT conditions and (ii) practical performance via accuracy assessment.

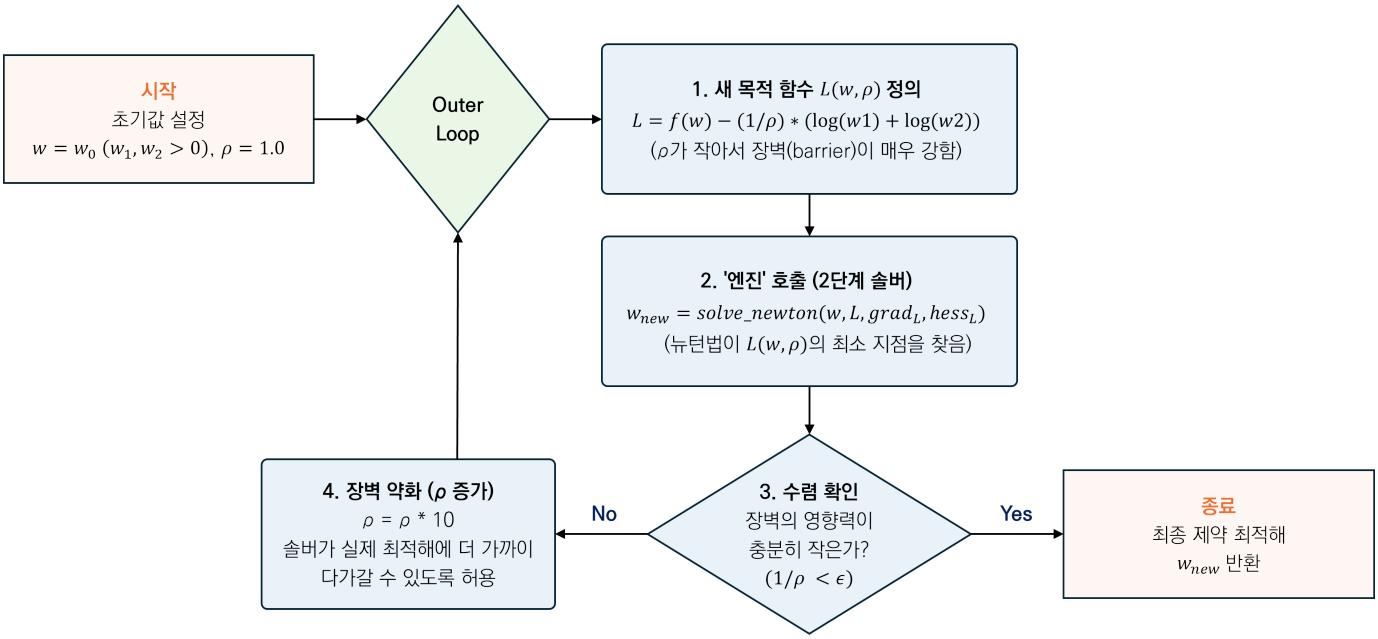


Fig. 3. Interior Point Method solver structure with outer-loop barrier updates and an inner Newton-based solver.

1) Verification 1: KKT Conditions: The Karush–Kuhn–Tucker (KKT) conditions constitute necessary optimality conditions for nonlinear programming problems. Verifying these conditions confirms whether the solver has found a mathematically valid local optimum [7].

For a problem of minimizing $f(\mathbf{x})$ subject to inequality constraints $g_i(\mathbf{x}) \leq 0$ and equality constraints $h_j(\mathbf{x}) = 0$, the KKT conditions are:

- *Stationarity:*

$$\nabla f(\mathbf{x}^*) + \sum_i \mu_i \nabla g_i(\mathbf{x}^*) + \sum_j \lambda_j \nabla h_j(\mathbf{x}^*) = \mathbf{0}. \quad (21)$$

- *Primal feasibility:* $g_i(\mathbf{x}^*) \leq 0$ and $h_j(\mathbf{x}^*) = 0$.
- *Dual feasibility:* $\mu_i \geq 0$ for all i.
- *Complementary slackness:* $\mu_i \, g_i(\mathbf{x}^*) = 0$ for all i.

Verification results: The numerical verification results for the toy-problem solution $\mathbf{w}^* = [582.03,\ 686.96,\ -2299.70]^\top$ are summarized below (within numerical tolerance).

a) *Stationarity:*

$$\nabla f(\mathbf{w}^*)_1 - \mu_1 = 7.23 \times 10^{-4} \ (\approx 0,\ \text{True}), \quad (22)$$
$$\nabla f(\mathbf{w}^*)_2 - \mu_2 = 5.27 \times 10^{-4} \ (\approx 0,\ \text{True}), \quad (23)$$
$$\nabla f(\mathbf{w}^*)_3 = 3.42 \times 10^{-4} \ (\approx 0,\ \text{True}). \quad (24)$$

b) *Primal feasibility:*

$$g_1(\mathbf{w}^*) = -w_1 = -5.82 \times 10^2 \ (\leq 0,\ \text{True}), \quad (25)$$
$$g_2(\mathbf{w}^*) = -w_2 = -6.87 \times 10^2 \ (\leq 0,\ \text{True}). \quad (26)$$

c) *Dual feasibility:*

$$\mu_1 = 0.00 \times 10^0 \ (\geq 0,\ \text{True}), \quad (27)$$
$$\mu_2 = 0.00 \times 10^0 \ (\geq 0,\ \text{True}). \quad (28)$$

d) *Complementary slackness:*

$$\mu_1 \, g_1(\mathbf{w}^*) = -0.00 \times 10^0 \ (\approx 0,\ \text{True}), \quad (29)$$
$$\mu_2 \, g_2(\mathbf{w}^*) = -0.00 \times 10^0 \ (\approx 0,\ \text{True}). \quad (30)$$

In conclusion, all four KKT conditions were satisfied within numerical tolerance. This provides mathematical evidence that the solver identified a valid constrained optimum.

2) Verification 2: Accuracy Evaluation: The dataset was split into a training set (80%) and a test set (20%). The constrained solver was trained using only the training set to obtain optimal weights $\mathbf{w}^*_{\text{train}}$, which were then used to predict labels for the unseen test set.

- *Optimal weights* ($\mathbf{w}^*_{train}$)*:* $[747.55,\ 608.57,\ -2362.15]^\top$
- *Actual labels:* $[1,\ 1,\ 0,\ 0,\ 0,\ 1]$
- *Predicted labels:* $[1,\ 1,\ 0,\ 1,\ 0,\ 1]$
- *Test accuracy:* 83.33%

The solver correctly classified 5 out of 6 test samples, achieving an accuracy of 83.33%. This indicates that the constrained optimization model learned a decision boundary and generalized to unseen data, supporting both mathematical robustness and practical efficacy.

IV. Real-World Application: UCI Breast Cancer Dataset

Following the successful validation of the constrained solver on synthetic data, the methodology was scaled to a high-dimensional, real-world classification problem. This section details the application of the generalized solver to the UCI Breast Cancer Wisconsin (Diagnostic) dataset [4].

A. Data Preparation and Model Redefinition

The transition from the toy problem to the real-world scenario necessitated a redefinition of the problem space

to accommodate higher dimensionality and domain-specific constraints.

1) Dataset and Preprocessing: The dataset comprises 569 samples with 30 numeric features computed from digitized images of a fine needle aspirate (FNA) of a breast mass.

- *Label encoding:* The original labels were retained (0 = Malignant, 1 = Benign).
- *Feature scaling:* Because the logarithmic barrier is sensitive to numerical scale, all 30 features were normalized to $[0, 1]$ using Min–Max scaling. This makes the barrier gradients comparable across dimensions.
- *Data splitting:* The dataset was partitioned into a training set (80%) and a test set (20%) using stratified sampling to preserve class distribution.
- *Bias term:* A bias term w_b was introduced by appending a column of ones to the feature matrix, increasing the dimensionality of the weight vector to $D = 31$:

$$\tilde{\mathbf{x}} = \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} \mathbf{w}_{1:30} \\ w_b \end{bmatrix}. \tag{31}$$

2) Constraint Design via Domain Knowledge: Unlike the toy problem, where constraints were arbitrary, constraints for this application were derived from medical domain knowledge regarding tumor characteristics. With $p = \sigma(\mathbf{w}^\top \tilde{\mathbf{x}})$ and $\tilde{\mathbf{x}} \in [0, 1]^{31}$ (hence $\tilde{\mathbf{x}} \geq \mathbf{0}$ elementwise after scaling), the sign of each weight controls whether increasing a feature increases or decreases the predicted probability.

The 30 feature weights were categorized into three groups to enforce logical consistency between feature values and diagnosis probability:

- *Group 1 ($w_i \leq 0$):* Features where larger values correlate with malignancy (label 0). To make $p = \sigma(\mathbf{w}^\top \tilde{\mathbf{x}})$ decrease as such a feature increases, the linear term $\mathbf{w}^\top \tilde{\mathbf{x}}$ should decrease, which is enforced by negative weights when $\tilde{\mathbf{x}} \geq \mathbf{0}$.
  Examples: radius, perimeter, area, concavity, concave points (and their *worst/error* variants).
  Constraint form:

$$w_i \leq 0 \iff g_i(\mathbf{w}) = w_i \leq 0, \tag{32}$$

  applied to 15 weights.
- *Group 2 ($w_j \geq 0$):* Features where larger values correlate with benignity (label 1). For p to increase with the feature value, $\mathbf{w}^\top \tilde{\mathbf{x}}$ should increase, which is enforced by positive weights when $\tilde{\mathbf{x}} \geq \mathbf{0}$.
  Examples: smoothness, symmetry, compactness, texture.
  Constraint form:

$$w_j \geq 0 \iff g_j(\mathbf{w}) = -w_j \leq 0, \tag{33}$$

  applied to 9 weights.
- *Group 3 (unconstrained):* Features with ambiguous or weak correlation, and the bias term.
  Examples: fractal dimension, texture error, and w_b.
  No constraints were applied (7 parameters).

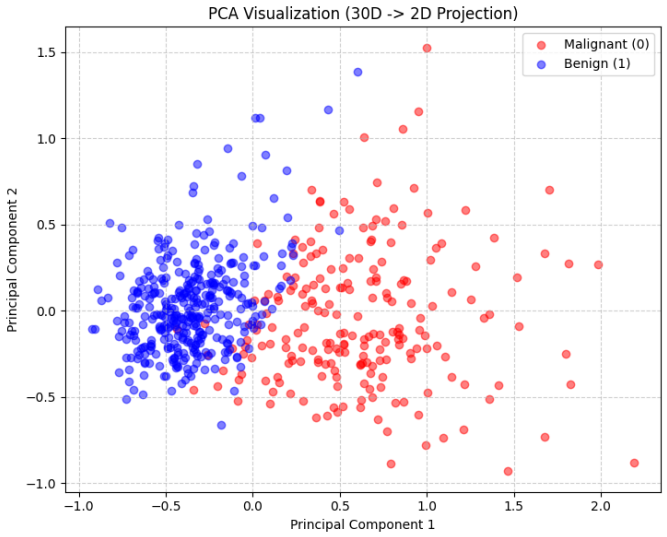


Fig. 4. PCA visualization of the UCI Breast Cancer Wisconsin (Diagnostic) dataset.

B. Generalization of the Solver Engine

To handle the 31-dimensional space with mixed inequality constraints ($w_i \leq 0$ and $w_j \geq 0$), the solver architecture was generalized from the hard-coded toy implementation.

1) Generalized Barrier Function: Let $\mathcal{G}_1$ be the index set for weights constrained as $w_i \leq 0$, and $\mathcal{G}_2$ be the index set for weights constrained as $w_j \geq 0$. Using the standard log-barrier form for inequalities $g(\mathbf{w}) \leq 0$, i.e., $-\log(-g(\mathbf{w}))$, the generalized surrogate objective is defined as

$$L(\mathbf{w}, \rho) = f(\mathbf{w}) - \frac{1}{\rho} \left(\sum_{i \in \mathcal{G}_1} \log(-w_i) + \sum_{j \in \mathcal{G}_2} \log(w_j) \right), \tag{34}$$

where $-w_i > 0$ for $i \in \mathcal{G}_1$ and $w_j > 0$ for $j \in \mathcal{G}_2$ must hold to keep the barrier well-defined.

a) Gradient: The barrier gradient components are added to $\nabla f(\mathbf{w})$. For constrained indices,

$$\frac{\partial L}{\partial w_k} = \frac{\partial f}{\partial w_k} - \frac{1}{\rho} \frac{1}{w_k}, \quad k \in \mathcal{G}_1 \cup \mathcal{G}_2, \tag{35}$$

and for unconstrained indices, $\frac{\partial L}{\partial w_k} = \frac{\partial f}{\partial w_k}$. Notably, if $k \in \mathcal{G}_1$ then $w_k < 0$ so $-\frac{1}{\rho w_k} > 0$, pushing w_k upward toward 0 from the negative side; if $k \in \mathcal{G}_2$ then $w_k > 0$ so $-\frac{1}{\rho w_k} < 0$, pushing w_k downward toward 0 from the positive side. In both cases, the barrier produces a repulsive force away from the feasibility boundary.

b) Hessian: The Hessian of L is obtained by adding a diagonal barrier-curvature term for constrained indices:

$$\nabla^2 L(\mathbf{w}, \rho) = \nabla^2 f(\mathbf{w}) + \frac{1}{\rho} \operatorname{diag}(\mathbf{c}), \tag{36}$$

where

$$c_k = \begin{cases} \frac{1}{w_k^2}, & k \in \mathcal{G}_1 \cup \mathcal{G}_2, \\ 0, & \text{otherwise.} \end{cases} \tag{37}$$

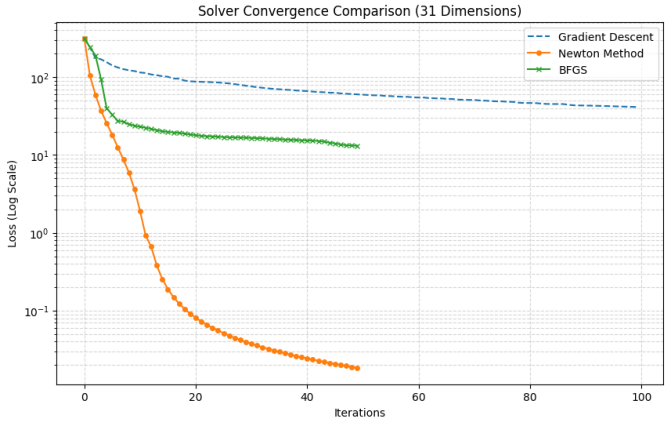


Fig. 5. Convergence comparison of GD, Newton's Method, and BFGS.

This creates extreme curvature near the boundary, naturally shrinking Newton steps as iterates approach infeasibility.

2) Robust Line Search: The line search was upgraded to perform a feasibility check across all constrained dimensions simultaneously. For a candidate step $\mathbf{w}_{\text{new}} = \mathbf{w} + \alpha \mathbf{d}$, the step is accepted only if it satisfies both feasibility and sufficient decrease:

$$\begin{aligned} \text{Feasibility:} \quad & (\mathbf{w}_{\text{new}})_i < -\epsilon, \ \forall i \in \mathcal{G}_1, \\ & (\mathbf{w}_{\text{new}})_j > \epsilon, \ \forall j \in \mathcal{G}_2. \end{aligned} \tag{38}$$

$$\text{Armijo:} \quad L(\mathbf{w}_{\text{new}}, \rho) \leq L(\mathbf{w}, \rho) + \beta \alpha \nabla L(\mathbf{w}, \rho)^\top \mathbf{d}. \tag{39}$$

If any single dimension violates (38) (e.g., a weight in $\mathcal{G}_1$ becomes non-negative, or a weight in $\mathcal{G}_2$ becomes non-positive), the step size α is immediately rejected and reduced, without evaluating the objective. This guarantees that the solver maintains the interior-point property throughout optimization.

C. Results and Verification

The generalized constrained solver was executed on the 31-dimensional training set. The optimization process was analyzed across three dimensions: computational convergence, mathematical optimality (KKT conditions), and predictive generalization.

1) Convergence Analysis: As illustrated in Fig. 5, the first-order Gradient Descent (GD) method exhibited slow, linear convergence, failing to minimize the loss significantly even after 100 iterations. In contrast, the second-order methods (Newton's Method and BFGS) leveraged curvature information to achieve near-quadratic convergence, reaching the minimum within approximately 20 iterations.

Table II summarizes the outer-loop behavior of the generalized IPM solver. The barrier tolerance parameter $(1/\rho)$ diminished to 2.40×10^{-7}, rendering the influence of the penalty term effectively negligible. Consequently, the feature weights stabilized at the constrained optimal solution, achieving a final loss of 22.1350.

TABLE II
CONVERGENCE OF THE FINAL CONSTRAINED SOLVER (UCI DATASET).

Outer Iter	ρ	$1/\rho$	Loss	Avg G1 (Neg)	Avg G2 (Pos)
0	10^0	2.40×10^1	36.3813	-23.4353	7.9642
1	10^1	2.40×10^0	23.3588	-12.2808	3.1578
2	10^2	2.40×10^{-1}	22.2741	-12.0746	2.7627
3	10^3	2.40×10^{-2}	22.1497	-12.1736	2.7146
4	10^4	2.40×10^{-3}	22.1365	-12.2095	2.7071
5	10^5	2.40×10^{-4}	22.1352	-12.2158	2.7062
6	10^6	2.40×10^{-5}	22.1351	-12.2165	2.7061
7	10^7	2.40×10^{-6}	22.1350	-12.2166	2.7061
8	10^8	2.40×10^{-7}	22.1350	-12.2166	2.7061

The stabilized constrained optimum is:

$$\begin{aligned} \mathbf{w}^* = [&-0.0000,\ 5.5731,\ -0.0000,\ -0.0000,\ 0.0000, \\ &22.4113,\ -4.9833,\ -18.1187,\ 0.0000,\ -2.7571, \\ &-0.0000,\ 13.3471,\ -0.0000,\ -109.9035,\ 0.0000, \\ &1.9433,\ -0.0000,\ -16.3574,\ 0.0000,\ 33.3453, \\ &-22.2841,\ -24.5892,\ -0.0000,\ -0.0000,\ 0.0000, \\ &0.0000,\ -3.2150,\ -8.3867,\ 0.0000,\ -24.8532, \\ &26.3813]^\top. \end{aligned} \tag{40}$$

2) Interpretability: Active vs. Inactive Constraints: To improve interpretability, we visualize the final constrained optimum $\mathbf{w}^*$ by grouping weights according to their constraint types (Group 1: $w \leq 0$, Group 2: $w \geq 0$, Group 3: unconstrained), as shown in Fig. 6.

This plot also reveals which constraints were *active* versus *inactive* at the solution. *Inactive constraints* correspond to weights with large magnitude in the permitted direction (e.g., a strongly negative weight in Group 1), indicating that the data naturally supports that sign even without a barrier. In contrast, *active constraints* appear as weights tightly clamped near zero, suggesting that the unconstrained optimizer would have preferred to violate the sign constraint, but the logarithmic barrier suppressed that tendency and enforced a domain-consistent parameterization. For example, weights such as *mean radius* (Group 1) or *mean smoothness* (Group 2) are driven close to zero, implying that the medical sign constraints explicitly removed contradictory effects and improved model consistency.

3) Optimization Dynamics: Example Trajectory Under a Sign Constraint: Fig. 7 visualizes the optimization trajectory projected onto two representative coordinates: the weight of *area error* (horizontal axis) and the weight of *mean texture* (vertical axis). The *area error* weight is constrained to remain non-positive ($w_{\text{area error}} \leq 0$), and the dashed vertical line at $w_{\text{area error}} = 0$ indicates the feasibility boundary. Starting from a safe interior initialization near $(0, 0)$, the trajectory never crosses into the infeasible region, demonstrating that the feasibility-aware line search and the logarithmic barrier successfully enforce the interior-point property.

The early iterations exhibit large, abrupt movements, which is characteristic of Newton-type methods that exploit curvature information to take aggressive steps toward the optimum. As the trajectory approaches the boundary, the barrier induces a strong repulsive effect, bending the path and implicitly shrinking step sizes to avoid infeasibility. In later iterations,

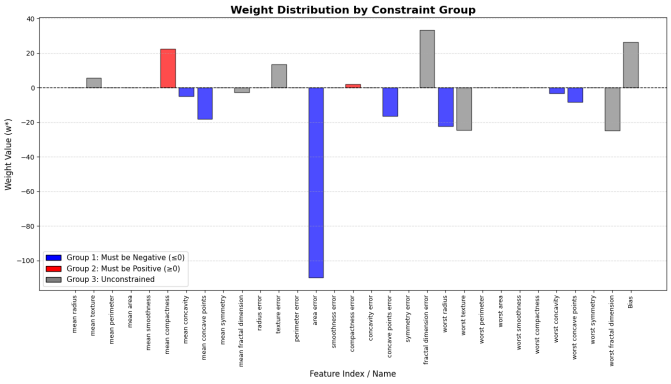


Fig. 6. Weight distribution by constraint group at the constrained optimum $\mathbf{w}^*$. Near-zero bars indicate *active* constraints (the barrier clamps the weight close to the boundary), whereas large-magnitude bars within the feasible sign indicate *inactive* constraints (the data already supports that sign).

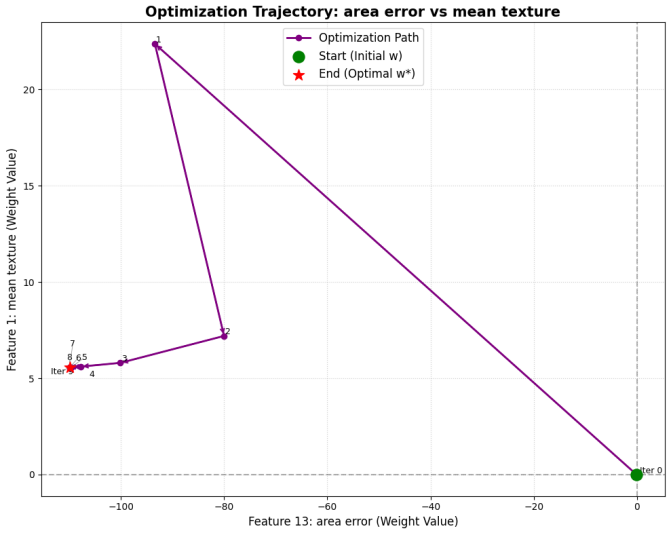


Fig. 7. Optimization trajectory projected onto (*area error*, *mean texture*) weight coordinates. The dashed line denotes the feasibility boundary for the sign constraint on *area error*. The path remains strictly in the feasible region after initialization, illustrating the interior-point behavior.

the solver performs fine-grained updates and stabilizes near the constrained optimum, confirming both rapid convergence and strict constraint satisfaction.

4) Verification 1: KKT Conditions: The mathematical validity of the solution $\mathbf{w}^*$ was verified against the Karush–Kuhn–Tucker (KKT) conditions under mixed inequality constraints, where Group 1 enforces $w_i \leq 0$ and Group 2 enforces $w_j \geq 0$ [7].

a) Primal feasibility ($g(\mathbf{w}^*) \leq 0$): The solver strictly enforced the domain constraints:

- *Group 1 ($w \leq \mathbf{0}$):* the maximum weight value was -9.06×10^{-8}, satisfying the non-positive constraint.
- *Group 2 ($w \geq \mathbf{0}$):* the minimum weight value was 1.71×10^{-8}, satisfying the non-negative constraint.

[KKT Conditions Verification Matrix]

	Idx	Feature	Group	Stationarity	Primal Feas.	Dual Feas.	Comp. Slack.	KKT Satisfied
0	0	mean radius	G1(<=0)	True	True	True	True	True
1	1	mean texture	G3(None)	True	True	True	True	True
2	2	mean perimeter	G1(<=0)	True	True	True	True	True
3	3	mean area	G1(<=0)	True	True	True	True	True
4	4	mean smoothness	G2(>=0)	True	True	True	True	True
5	5	mean compactness	G2(>=0)	True	True	True	True	True
6	6	mean concavity	G1(<=0)	True	True	True	True	True
7	7	mean concave points	G1(<=0)	True	True	True	True	True
8	8	mean symmetry	G2(>=0)	True	True	True	True	True
9	9	mean fractal dimension	G3(None)	True	True	True	True	True
10	10	radius error	G1(<=0)	True	True	True	True	True
11	11	texture error	G3(None)	True	True	True	True	True
12	12	perimeter error	G1(<=0)	True	True	True	True	True
13	13	area error	G1(<=0)	True	True	True	True	True
14	14	smoothness error	G2(>=0)	True	True	True	True	True
15	15	compactness error	G2(>=0)	True	True	True	True	True
16	16	concavity error	G1(<=0)	True	True	True	True	True
17	17	concave points error	G1(<=0)	True	True	True	True	True
18	18	symmetry error	G2(>=0)	True	True	True	True	True
19	19	fractal dimension error	G3(None)	True	True	True	True	True
20	20	worst radius	G1(<=0)	True	True	True	True	True
21	21	worst texture	G3(None)	True	True	True	True	True
22	22	worst perimeter	G1(<=0)	True	True	True	True	True
23	23	worst area	G1(<=0)	True	True	True	True	True
24	24	worst smoothness	G2(>=0)	True	True	True	True	True
25	25	worst compactness	G2(>=0)	True	True	True	True	True
26	26	worst concavity	G1(<=0)	True	True	True	True	True
27	27	worst concave points	G1(<=0)	True	True	True	True	True
28	28	worst symmetry	G2(>=0)	True	True	True	True	True
29	29	worst fractal dimension	G3(None)	True	True	True	True	True
30	30	Bias	G3(None)	True	True	True	True	True

Fig. 8. KKT condition verification summary across constrained feature weights.

b) Stationarity and dual feasibility: The stationarity condition was examined for both active and inactive constraints:

$$\nabla f(\mathbf{w}^*) + \sum_k \mu_k \nabla g_k(\mathbf{w}^*) = \mathbf{0}, \qquad \mu_k \geq 0. \tag{41}$$

Empirically, the following behaviors were observed:

- *Inactive constraints ($|w_k^*| \gg 0$):* for features whose optimal weights were naturally non-zero, the corresponding gradient components $\nabla f(\mathbf{w}^*)_k$ converged close to zero, and the Lagrange multipliers were effectively $\mu_k \approx 0$.
- *Active constraints ($w_k^* \approx 0$):* for features suppressed by the constraints:
  - For Group 1 (constraint $w_i \leq 0$), negative gradient components ($\nabla f(\mathbf{w}^*)_i < 0$) indicate that the unconstrained descent direction would increase w_i toward positive values. The KKT multiplier satisfies $\mu_i > 0$ to counteract this tendency and maintain feasibility.
  - For Group 2 (constraint $w_j \geq 0$, equivalently $-w_j \leq 0$), positive gradient components ($\nabla f(\mathbf{w}^*)_j > 0$) indicate that the unconstrained descent direction would decrease w_j toward negative values. The KKT multiplier $\mu_j > 0$ provides the opposing force required to keep w_j non-negative.

  In all cases, dual feasibility $\mu_k \geq 0$ was satisfied.

c) Complementary slackness: For all constrained dimensions, the complementary slackness products were verified to be negligible:

$$\mu_k \, g_k(\mathbf{w}^*) \approx 0, \tag{42}$$

confirming that no constraint was simultaneously inactive and enforced with a non-zero multiplier.

As shown in Fig. 8, the solver satisfied the necessary conditions for optimality across the constrained feature set.

5) Verification 2: Accuracy Evaluation: The practical utility of the derived solution $\mathbf{w}^*$ was evaluated by measuring classification accuracy on the training and test sets. The final

weight vector represents a balance between minimizing the negative log-likelihood and adhering to domain-driven medical constraints.

- *Train set accuracy:* 97.80%
- *Test set accuracy:* 98.25%

The model achieved 98.25% accuracy on the unseen test set, exceeding the training accuracy. This suggests that the domain-specific constraints acted as a form of regularization: by preventing physically implausible weight signs (e.g., a positive weight for tumor radius under the chosen label convention), the constraints likely reduced overfitting to noise in the training data. Overall, the constrained optimization solver not only produced a mathematically valid solution but also yielded a robust and highly generalizable predictive model for medical diagnosis.

V. Conclusion

In this work, we designed and implemented a robust constrained optimization solver capable of handling mixed inequality constraints in high-dimensional spaces. By coupling the Interior Point Method with a Newton-based search engine, the solver bridges theoretical optimization principles and practical deployment.

The experimentation yielded three key findings:

- **Algorithmic efficiency:** The comparative analysis confirmed that second-order methods (Newton's Method and BFGS) significantly outperform first-order Gradient Descent in convergence speed and stability [1], [7]. Our IPM solver maintained this efficiency, converging rapidly even in a 31-dimensional space with barrier-induced curvature.
- **Mathematical rigor:** The solver demonstrated strict adherence to constraints. In the real-world application, the mathematical validity of the optimal solution $\mathbf{w}^*$ was verified via the Karush–Kuhn–Tucker (KKT) conditions: stationarity, primal feasibility, dual feasibility, and complementary slackness were satisfied within numerical tolerances [7].
- **Generalization via constraints:** The constrained model achieved a test-set accuracy of 98.25%, exceeding the training-set accuracy of 97.80%. This suggests that enforcing domain-specific constraints acted as a potent form of regularization. By restricting the hypothesis space to medically valid parameter signs, the solver likely reduced overfitting to noise and improved generalization on unseen data [3], [6].

In conclusion, this project validates that optimization guided by domain knowledge can yield models that are interpretable, feasible, and strong in predictive performance. Future work may explore extending the solver to non-convex settings or integrating quasi-Newton updates within the barrier framework to further reduce computational cost.

References

[1] J. Nocedal and S. J. Wright, *Numerical Optimization*. New York, NY, USA: Springer, 2006.

[2] A. Giuliani and A. Candelieri, "Integration between constrained optimization and deep networks: A survey," *Frontiers in Artificial Intelligence*, vol. 7, Art. no. 1414707, 2024.

[3] C. Rudin, "Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead," *Nature Machine Intelligence*, vol. 1, no. 5, pp. 206–215, 2019.

[4] W. Wolberg, O. Mangasarian, N. Street, and W. Street, "Breast Cancer Wisconsin (Diagnostic) Data Set," UCI Machine Learning Repository, 1995, doi: 10.24432/C5DW2B.

[5] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York, NY, USA: Springer, 2006.

[6] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge, U.K.: Cambridge University Press, 2004.

[7] M. J. Kochenderfer and T. A. Wheeler, *Algorithms for Optimization*. Cambridge, MA, USA: MIT Press, 2019.